

HLD

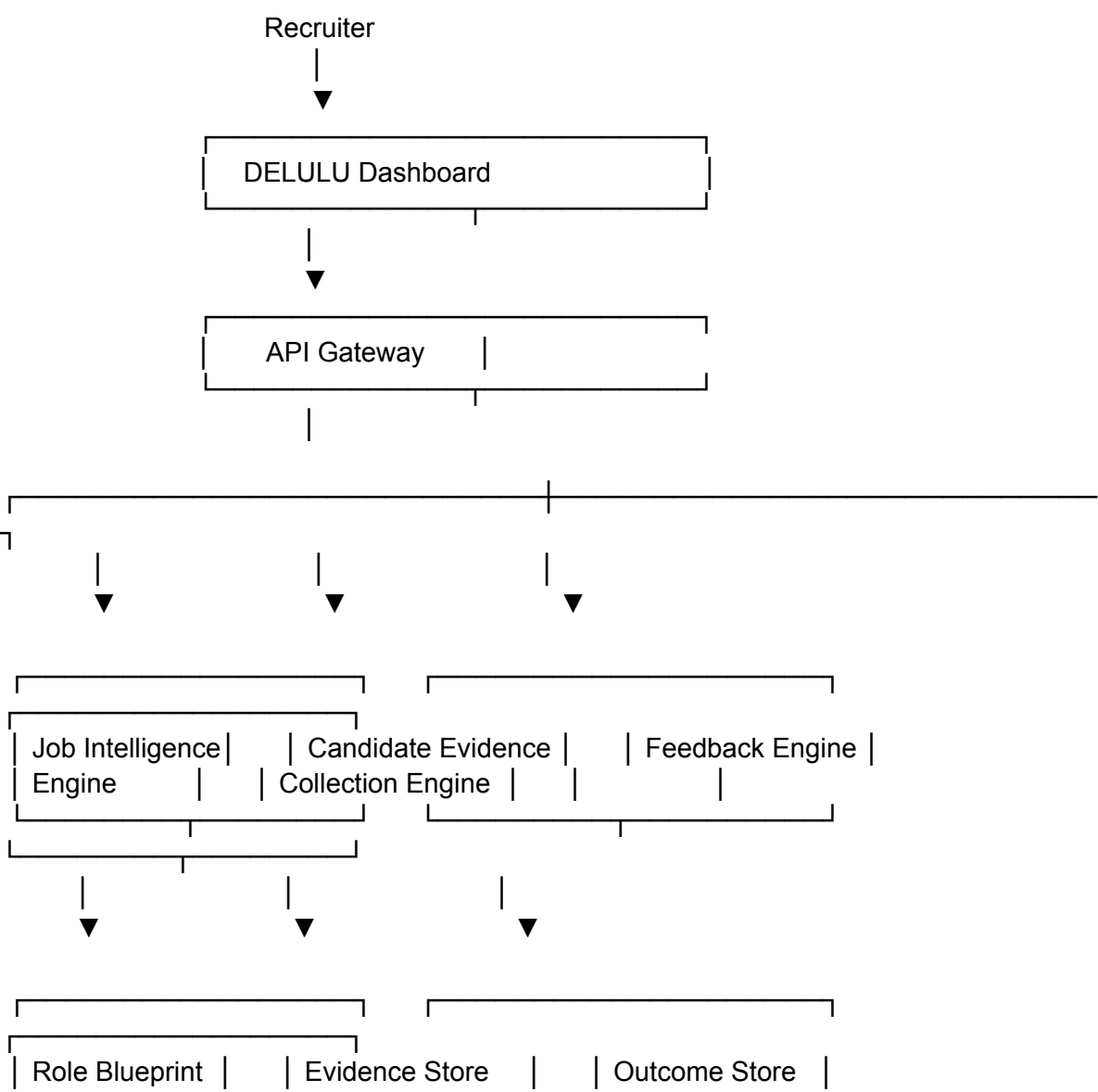
DELULU Final Vision HLD

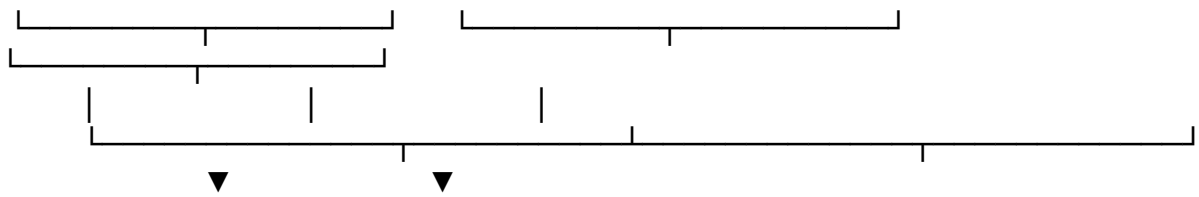
AI Hiring Intelligence Platform

Mission

Discover high-potential candidates overlooked by traditional ATS systems through evidence-driven, explainable hiring intelligence.

Full Product HLD





Relevance Filter Engine



Signal Extraction Engine



Capability Engine



Risk Engine



Confidence Engine

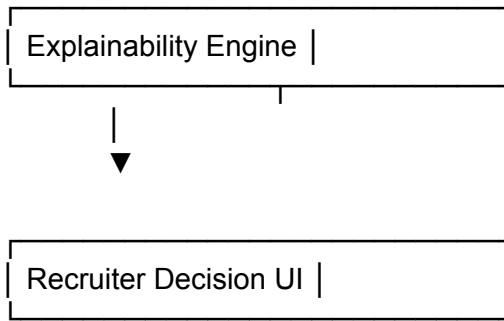


Hidden Talent Engine



Ranking Engine





Core Domain Objects

These become the heart of DELULU.

Role Blueprint

Generated from JD.

```
{
  "role": "AI Engineer",
  "skills": [...],
  "behavioral_traits": [...],
  "weights": {...},
  "required_evidence": [...]
}
```

Candidate Profile

```
{
  "candidate_id": "",
  "resume": "",
  "linkedin": "",
  "github": "",
  "evidence": []
}
```

Capability Profile

```
{  
  "technical":88,  
  "execution":91,  
  "ownership":90,  
  "learning_velocity":94,  
  "problem_solving":86,  
  "domain_expertise":82  
}
```

Risk Profile

```
{  
  "evidence_risk":12,  
  "execution_risk":18,  
  "credibility_risk":15,  
  "role_gap_risk":22  
}
```

Hidden Talent Profile

```
{  
  "visibility":25,  
  "capability":89,  
  "hti":64  
}
```

Final Recommendation

```
{  
  "fit_score":91,  
  "confidence":87,  
  "risk":18,  
  "hti":64,  
  "recommendation":"Interview"  
}
```

Major Engines

1. Job Intelligence Engine

Input:

JD

Output:

Role Blueprint

Responsibilities:

- Skill extraction
 - Behavioral trait extraction
 - Weight generation
 - Role understanding
-

2. Evidence Collection Engine

Sources:

Resume
LinkedIn
GitHub
Portfolio

Future:

Kaggle
Codeforces
ATS
Workday

3. Relevance Filter Engine

Purpose:

Remove role-irrelevant noise.

Example:

AI Role

Keep:

ClinicBot

Forge

WattWise

Discard:

HTML Assignment

Calculator App

4. Capability Engine

Computes:

Technical

Execution

Ownership

Learning

Problem Solving

Domain

5. Risk Engine

Computes:

Evidence Risk

Execution Risk

Role Gap Risk

Credibility Risk

Growth Risk

Consistency Risk

6. Confidence Engine

Computes:

How trustworthy is this score?

Inputs:

Evidence Quality

Evidence Quantity

Cross-source agreement

7. Hidden Talent Engine

Computes:

Visibility

Capability

HTI

Purpose:

Find overlooked candidates.

8. Ranking Engine

Produces:

Final Candidate Ordering

9. Explainability Engine

Generates:

Why selected

Why rejected

Strengths

Weaknesses

Missing Skills

10. Feedback Engine

Stores:

Recruiter actions

Shortlisted

Rejected

Interviewed

Hired

Used for future learning.

Technology HLD

Frontend

- |
- |— Next.js
- |— Tailwind
- |— Shadcn
- |— Recharts

Backend

- |
- |— FastAPI
- |— Python
- |— Celery
- |— Redis

AI Layer

- |
- |— Gemini
- |— Sentence Transformers

└─ BGE Embeddings

Storage

└─ PostgreSQL
└─ Qdrant
└─ Redis

MVP HLD

Now let's remove all future complexity.

This is what you should actually build.

MVP Goal

JD

↓

Candidate Profiles

↓

Ranking

↓

Hidden Talent Discovery

↓

Explanation

That's it.

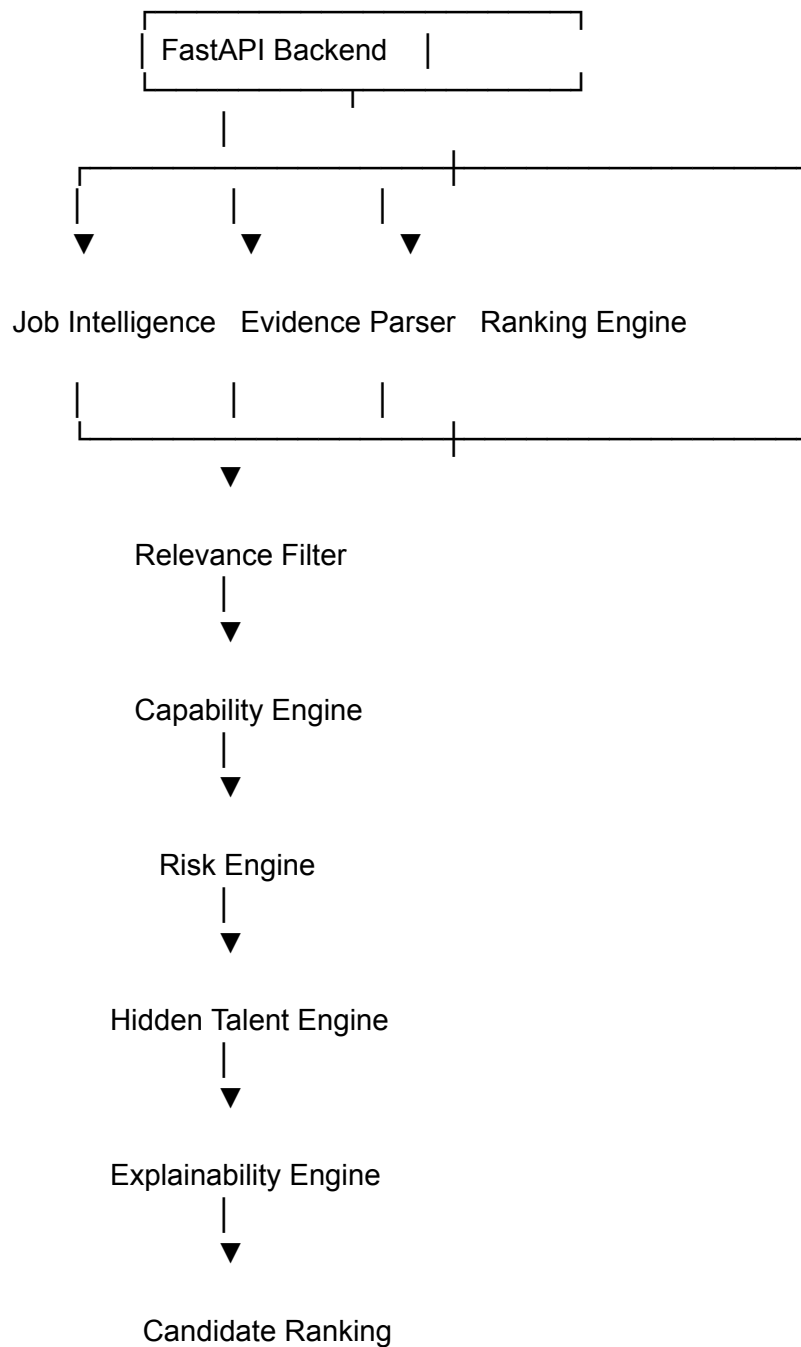
MVP Architecture

Recruiter



DELULU Dashboard |





MVP Modules

Build

✓ JD Parser

✓ Resume Parser

- ✓ GitHub Parser
 - ✓ LinkedIn Parser
 - ✓ Relevance Filter
 - ✓ Capability Engine
 - ✓ Risk Engine
 - ✓ HTI Engine
 - ✓ Ranking Engine
 - ✓ Explainability Engine
 - ✓ Dashboard
-

Skip

- ✗ Neo4j
 - ✗ Multi-Agent Debate
 - ✗ Digital Twin
 - ✗ ATS Integrations
 - ✗ Continuous Learning
 - ✗ Hiring Outcome Prediction
 - ✗ Enterprise Connectors
 - ✗ Team Composition Engine
 - ✗ Company DNA Engine
-

MVP Database

PostgreSQL

jobs
candidates
candidate_evidence
role_blueprints

scores
rankings

Qdrant

candidate_embeddings
job_embeddings

Redis

cache
background jobs

Hackathon Demo Flow

Upload JD
↓
Upload 20 Candidates
↓
DELULU analyzes evidence
↓
Creates capability profiles
↓
Computes risk
↓
Computes HTI
↓
Ranks candidates
↓
Explains ranking
↓
Surfaces hidden talent

LLD

Before LLD, one warning:

Your current HLD contains about **10 services**. For a hackathon, that is suicide. Your LLD should not be microservices.

Build a **modular monolith**.

DELULU MVP LLD

Frontend (Next.js)

|



FastAPI Backend

- ├── Job Module
- ├── Candidate Module
- ├── Evidence Module
- ├── Scoring Module
- ├── Explainability Module
- └── Ranking Module

|



Postgres

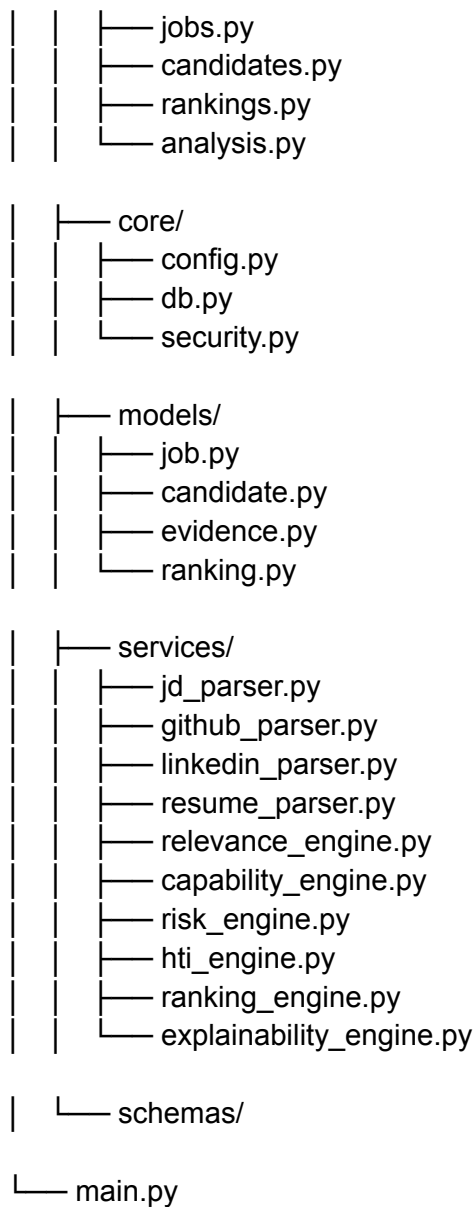
Qdrant

Folder Structure

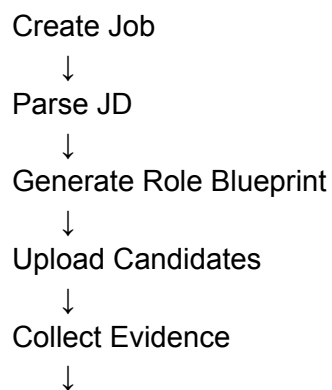
backend/

├── app/

| └── api/



Core Workflow



Relevance Filtering



Signal Extraction



Capability Score



Risk Score



HTI Score



Ranking



Explanation

Database LLD

jobs

jobs

id UUID PK

title TEXT

description TEXT

role_blueprint JSONB

created_at TIMESTAMP

candidates

candidates

id UUID PK

job_id UUID

name TEXT

email TEXT

resume_url TEXT

linkedin_url TEXT

github_url TEXT

created_at TIMESTAMP

candidate_evidence

candidate_evidence

id UUID PK

candidate_id UUID

source_type TEXT

source_url TEXT

raw_content JSONB

processed_content JSONB

relevance_score FLOAT

capability_profiles

capability_profiles

id UUID PK

candidate_id UUID

technical FLOAT

execution FLOAT

ownership FLOAT

learning_velocity FLOAT

problem_solving FLOAT

domain_expertise FLOAT

capability_score FLOAT

risk_profiles

risk_profiles

id UUID PK

candidate_id UUID

evidence_risk FLOAT

role_gap_risk FLOAT

credibility_risk FLOAT

risk_score FLOAT

hidden_talent_profiles

hidden_talent_profiles

id UUID PK

candidate_id UUID

visibility_score FLOAT

hti_score FLOAT

rankings

rankings

id UUID PK

job_id UUID

candidate_id UUID

fit_score FLOAT

confidence FLOAT

rank INTEGER

recommendation TEXT

Module 1

Job Intelligence Engine

Input:

```
{  
  "job_description": "Senior AI Engineer..."  
}
```

Output

```
{  
  "skills":[  
    "Python",  
    "LLMs",  
    "FastAPI"  
  ],  
  "traits":[  
    "Ownership",  
    "Execution",  
    "Learning"  
  ],  
  "weights":{  
    "technical":0.35,  
    "execution":0.25,  
    "ownership":0.20,  
    "learning":0.20  
  }  
}
```

```
}
```

Job API

POST /jobs

Response

```
{  
  "job_id": "123"  
}
```

Module 2

Candidate Evidence Engine

Sources

Resume

GitHub

LinkedIn

API

POST /candidates

Payload

```
{  
  "job_id": "123",  
  
  "name": "Venu",  
  
  "resume": "resume.pdf",  
}
```

```
"github":"github.com/venu",  
  
"linkedin":"linkedin.com/venu"  
}
```

Resume Parser

Extract:

```
{  
  "skills":[],  
  "projects":[],  
  "experience":[],  
  "education":[]  
}
```

GitHub Parser

Extract:

```
{  
  "repositories":[],  
  "languages":[],  
  "commit_stats":[],  
  "stars":[],  
  "deployments":[]  
}
```

LinkedIn Parser

Extract:

```
{  
  "experience":[],  
  "skills":[],  
  "certifications":[]  
}
```

```
}
```

Module 3

Relevance Engine

Most important module.

Input

```
{  
  "role": "AI Engineer",  
  
  "evidence": [  
    "ClinicBot",  
    "HTML Lab",  
    "Forge"  
  ]  
}
```

Prompt

Rate relevance from 0-100.

Role:
AI Engineer

Evidence:
ClinicBot

Explain.

Output

```
{  
  "artifact": "ClinicBot",  
  "relevance": 94  
}
```

Store only:

relevance > 60

Module 4

Capability Engine

Computes

Technical

Execution

Ownership

Learning Velocity

Technical

Input

Projects

Skills

GitHub

Output

```
{  
  "technical":87  
}
```

Execution

Input

Deployments

Products

Production Systems

Output

```
{  
  "execution":91  
}
```

Ownership

Input

Repo Creator

Lead Contributor

Founder Projects

Output

```
{  
  "ownership":89  
}
```

Learning

Input

Technology Timeline

Output

```
{  
  "learning":94  
}
```

Final

Capability

=

0.30 Technical

+

0.30 Execution

+

0.20 Ownership

+

0.20 Learning

Module 5

Risk Engine

Computes

Evidence Risk

Role Gap Risk

Credibility Risk

Evidence Risk

Missing sources

Low evidence

Weak validation

Role Gap

Missing required skills

Credibility

Claims != Evidence

Output

```
{  
  "risk":18  
}
```

Module 6

Hidden Talent Engine

Computes:

Visibility Score

Sources

College Prestige

Brand Companies

Awards

Public Recognition

Capability

Already computed.

HTI

HTI

=

Capability

-

Visibility

Normalize:

0-100

Output

```
{  
  "visibility":22,  
  "hti":68  
}
```

Module 7

Ranking Engine

Input

Capability

Risk

HTI

Confidence

Formula

Fit Score

=

0.55 Capability

+

0.25 HTI

+

0.20 Confidence

-

0.15 Risk

Output

```
{  
  "fit_score":91  
}
```

Module 8

Explainability Engine

Input

```
{  
  "candidate":"Venu",  
  
  "capability":89,  
  
  "risk":18,  
  
  "hti":66  
}
```

Output

```
{
  "strengths":[
    "Strong execution",
    "High ownership",
    "Rapid learning"
  ],

  "risks":[
    "Limited enterprise scale"
  ],

  "reason":
  "Candidate shows strong evidence of AI system delivery with high learning velocity."
}
```

Frontend LLD

Page 1

Job Upload

Paste JD

Create Job

Page 2

Candidate Upload

Resume

GitHub

LinkedIn

Page 3

Rankings

Rank

Fit Score

HTI

Risk

Confidence

Example:

#1 Venu

Fit:91

HTI:66

Risk:18

Page 4

Candidate Detail

Capability Radar

Risk Breakdown

Explanation

Evidence

PITCH

Pitch Narrative

The winning narrative is:

Hiring is optimized for credentials.

Talent is distributed unevenly.

Traditional ATS finds visible candidates.

DELULU finds capable candidates.

Not by resumes.

By evidence.

Everything should support that.

Final Deck Structure (15 Slides)

Slide 1 — Opening

DELULU

Hiring Intelligence Platform

Tagline:

We don't rank resumes. We rank evidence.

Subline:

Discover high-potential candidates overlooked by traditional ATS systems.

Slide 2 — The Problem

Hiring is broken before interviews begin

Visual:

1000 Applications

↓ ATS

980 Rejected

↓ Recruiter

20 Reviewed

Question:

What if the best candidate never reaches the interview?

Slide 3 — Why ATS Fails

Traditional ATS optimizes for:

- Keywords
- Brand names
- College prestige
- Resume writing skills

Not:

- Execution
- Ownership
- Learning ability
- Potential

Visual comparison.

Slide 4 — The Hidden Talent Problem

Candidate A

MIT

Google

OpenAI

Candidate B

Unknown college
Built real systems
Consistent growth

ATS:

A wins

DELULU:

Let's examine evidence

Introduce:

Hidden Talent Index (HTI)

Slide 5 — Our Solution

DELULU

AI Hiring Intelligence Platform

Input:

JD
Resume
LinkedIn
GitHub
Portfolio

Output:

Capability
Risk
Confidence
HTI
Recommendation

Slide 6 — How DELULU Thinks

Visual:

Job Description



Role Blueprint



Evidence Collection



Capability Analysis



Risk Analysis



Hidden Talent Discovery



Ranking

Simple architecture diagram.

Slide 7 — Role Intelligence

Problem:

AI Engineer

means different things everywhere.

DELULU creates:

Role Blueprint

Example:

Skills:

Python

RAG

Agents

Traits:

Ownership

Execution

Learning

Slide 8 — Evidence > Resumes

Candidate evidence sources:

Resume
GitHub
LinkedIn
Portfolio

Then:

Relevance Filtering

Example:

AI Role

Keep:

ClinicBot
Forge
WattWise

Ignore:

HTML Lab
Calculator

This is a very important slide.

Judges will remember it.

Slide 9 — Capability Engine

Dimensions:

Technical Competence

Execution

Ownership

Learning Velocity

Visual radar chart.

Example candidate profile.

Slide 10 — Risk Engine

Question:

Why should I NOT hire this person?

Risk Types:

Evidence Risk

Role Gap Risk

Credibility Risk

Output:

Capability = 91

Risk = 18

Slide 11 — Hidden Talent Engine

This is the hero slide.

Definition:

Hidden Talent = High Capability + Low Visibility

Visual:

2-axis graph

Capability ↑

Hidden Talent



Visibility →

Show:

Candidate #12

Capability: 89

Visibility: 23

HTI: 66

Slide 12 — Live Demo

Workflow:

Upload JD

↓

Upload Candidates

↓

Analyze Evidence

↓

Discover Hidden Talent

↓

Explain Ranking

Short GIF or screenshots.

Slide 13 — Results

This is mandatory.

Compare:

ATS	DELULU
Keyword Match	Evidence Match
Credentials	Capability
Opaque	Explainable
Resume Focused	Multi-Signal
Visible Talent	Hidden Talent

Slide 14 — Why DELULU Wins

Recruiters get

- Faster screening
- Explainable recommendations
- Hidden talent discovery
- Reduced bias toward prestige

Companies get

- Better candidate discovery
 - Better interview prioritization
-

Slide 15 — Closing

Big statement:

The best candidates aren't always the most visible.

DELULU helps recruiters discover the talent traditional ATS systems overlook.

Then:

We don't rank resumes.

We rank evidence.

Demo Script (Very Important)

Most teams show architecture.

Winning teams show transformation.

Do this:

Candidate A

Top College

Big Company

Weak Evidence

Candidate B

Unknown College

Strong Evidence

Strong Learning Velocity

Run DELULU.

Show:

ATS Rank = #15

DELULU Rank = #2

Judge instantly understands.

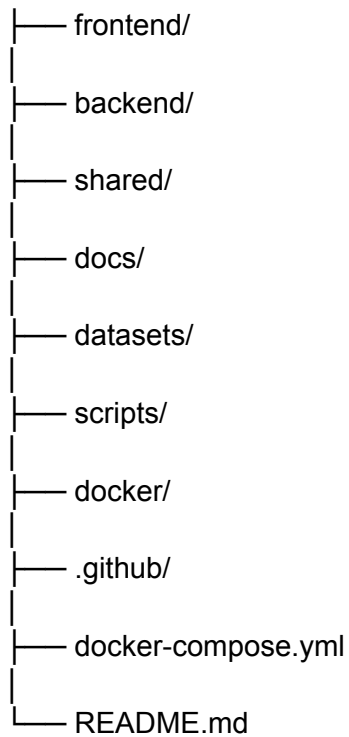
REPO & DEV

Repository Structure

Do NOT create separate repos.

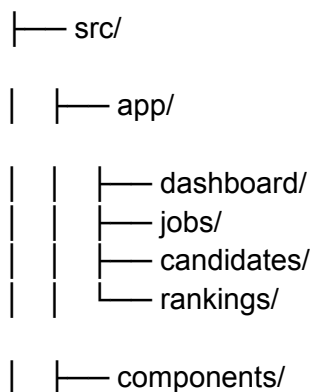
Use a monorepo.

delulu/



Frontend

frontend/



```
| | | jobs/
| | | candidates/
| | | ranking/
| | | charts/
```

```
| | | lib/
| | | | api.ts
| | | | types.ts
```

```
| | | hooks/
```

```
| | public/
| | package.json
```

Tech:

Next.js 16
TypeScript
Tailwind
shadcn/ui
Recharts

Backend

backend/

```
| | app/
```

```
| | | api/
```

```
| | | | jobs.py
| | | | candidates.py
| | | | rankings.py
| | | | analysis.py
```

```
| | | core/
```

```
| | | | config.py
| | | | database.py
| | | | security.py
```

```
| | | models/
```

```
| |— schemas/
| |— services/
| | |— jd/
| | |— evidence/
| | |— capability/
| | |— risk/
| | |— hti/
| | |— ranking/
| |— workers/
| |— main.py
|— tests/
|— requirements.txt
```

Shared

Common types.

shared/

```
|— role_blueprint.json
|— candidate_schema.json
|— ranking_schema.json
|— prompt_templates/
```

Docs

docs/

```
|— hld.md
|— lld.md
```

|— scoring.md

|— api-contracts.md

|— architecture.png

Database Schema (SQLAlchemy)

jobs

```
class Job(Base):
    __tablename__ = "jobs"

    id = Column(UUID)

    title = Column(String)

    description = Column(Text)

    role_blueprint = Column(JSON)

    created_at = Column(DateTime)
```

candidates

```
class Candidate(Base):
    __tablename__ = "candidates"

    id = Column(UUID)

    job_id = Column(UUID)

    name = Column(String)

    email = Column(String)

    linkedin_url = Column(String)
```

```
github_url = Column(String)

resume_path = Column(String)

created_at = Column(DateTime)
```

evidence

Stores parsed information.

```
class Evidence(Base):
    __tablename__ = "evidence"

    id = Column(UUID)

    candidate_id = Column(UUID)

    source_type = Column(String)

    raw_data = Column(JSON)

    processed_data = Column(JSON)

    relevance_score = Column(Float)
```

capability_profiles

```
class CapabilityProfile(Base):
    __tablename__ = "capability_profiles"

    candidate_id = Column(UUID)

    technical = Column(Float)

    execution = Column(Float)

    ownership = Column(Float)

    learning_velocity = Column(Float)

    capability_score = Column(Float)
```

risk_profiles

```
class RiskProfile(Base):
    __tablename__ = "risk_profiles"

    candidate_id = Column(UUID)

    evidence_risk = Column(Float)

    role_gap_risk = Column(Float)

    credibility_risk = Column(Float)

    risk_score = Column(Float)
```

hidden_talent_profiles

```
class HiddenTalentProfile(Base):
    __tablename__ = "hidden_talent_profiles"

    candidate_id = Column(UUID)

    visibility_score = Column(Float)

    hti_score = Column(Float)
```

rankings

```
class Ranking(Base):
    __tablename__ = "rankings"

    candidate_id = Column(UUID)

    job_id = Column(UUID)

    fit_score = Column(Float)
```


confidence = Column(Float)

recommendation = Column(String)

rank = Column(Integer)

API Contracts

Create Job

POST /api/jobs

Request

```
{  
  "title": "AI Engineer",  
  "description": "..."  
}
```

Response

```
{  
  "job_id": "uuid"  
}
```

Upload Candidate

POST /api/candidates

FormData

resume.pdf

github_url

linkedin_url

job_id

Response

```
{
  "candidate_id":"uuid"
}
```

Analyze Candidate

POST /api/candidates/{id}/analyze

Response

```
{
  "status":"completed"
}
```

Get Rankings

GET /api/jobs/{job_id}/rankings

Response

```
[
  {
    "candidate":"Venu",
    "fit_score":91,
    "risk":18,
    "hti":66,
    "confidence":92
  }
]
```

Candidate Detail

GET /api/candidates/{id}

Response

```
{
  "capability": {},
  "risk": {},
  "hti": {},
  "evidence": [],
  "explanation": {}
}
```

Team Ownership Map

This is extremely important.

Member 1

AI Core Lead

Own:

JD Parser

Role Blueprint

Capability Engine

Risk Engine

HTI Engine

Ranking Engine

Branches:

feat/job-intelligence

feat/scoring-engine

feat/ranking

Member 2

Data Ingestion Lead

Own:

Resume Parser

GitHub Parser

LinkedIn Parser

Evidence Store

Branches:

feat/evidence

feat/github-parser

feat/resume-parser

Member 3

Frontend Lead

Own:

Dashboard

Ranking Table

Candidate Detail

Charts

Branches:

feat/frontend

feat/dashboard

Member 4

Evaluation & Demo Lead

Own:

Dataset Creation

ATS Baseline

Metrics

Pitch Deck

Demo Script

Branches:

feat/evaluation

feat/demo

Demo Dataset Strategy

This is where most teams fail.

Do NOT use random resumes.

Create controlled candidates.

Job

Use one role.

AI Engineer

Only.

Do not build 10 roles.

Candidate Set

Create:

20 candidates

Group A

Obvious Strong Candidates

MIT

Google

OpenAI

5 candidates.

Group B

Average Candidates

Normal resumes

10 candidates.

Group C

Hidden Talent Candidates

This is DELULU's showcase.

5 candidates.

Example:

Unknown college

Strong GitHub

Strong projects

Strong growth trajectory

No prestige

ATS ranks:

#15

DELULU ranks:

#2

This becomes your demo moment.

ATS Baseline

Implement a dumb ATS.

```
score =  
keyword_match  
+  
years_experience  
+  
college_score
```

Use it for comparison.

Evaluation Metrics

Show:

Recruiter Agreement

Hidden Talent Recall

Average Candidate Score

ATS vs DELULU Difference

Most Important Deliverable

Not the backend.

Not the frontend.

Not Gemini.

The most important artifact is:

candidate_dataset_v1.json

Because without a carefully designed dataset, DELULU cannot demonstrate its value.